



**Yenepoya Institute of
Arts, Science, Commerce, and Management
Project – low Level Design
on
Natural Language Interface for Agriculture
Infrastructure**

Student Name: Aswanth M O

Guided By: Mr. Praveen

Industry Mentor: Mr. Shashank

Table of Contents

1. Introduction.
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. System Design.
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue.
3. Data Design.
 - 3.1. Data Model
 - 3.2. Data Access Mechanism
 - 3.3. Data Retention Policies
 - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
 - 7.1. Security Aspects
 - 7.2. Performance Aspects
8. References

1. Introduction

1.1 Scope

This document explains the internal working of the Agriculture Chatbot, including modules, request flow, data handling, and session behavior.

1.2 Intended Audience

- Developers
- Testers
- Maintainers
- Reviewers

1.3 System Overview

The system includes:

- Python web server
- Shared chatbot logic
- Static agriculture data
- Website frontend
- CLI version

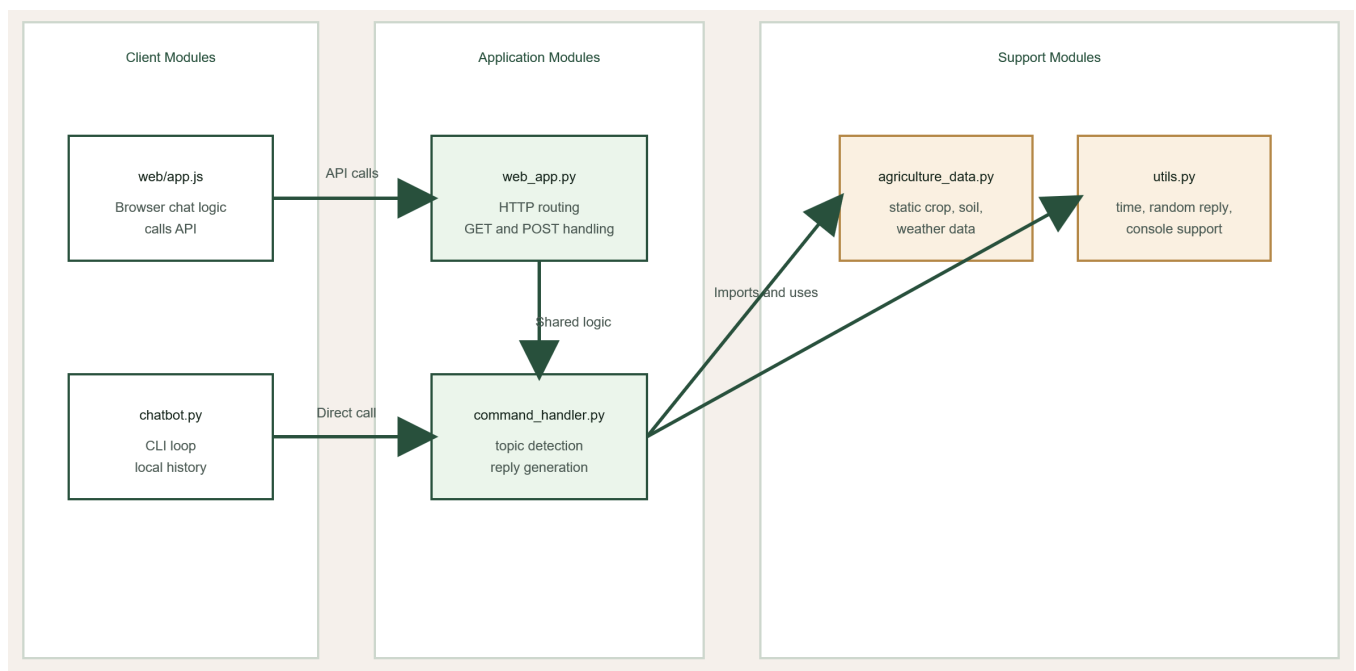
Both web and CLI use the same logic for consistent behavior.

2. System Design

2.1 Application Design

Module	Role
web_app.py	Handles HTTP requests/responses
command_handler.py	Processes user input
agriculture_data.py	Stores chatbot data
utils.py	Helper functions
chatbot.py	CLI chatbot
web/app.js	Frontend logic

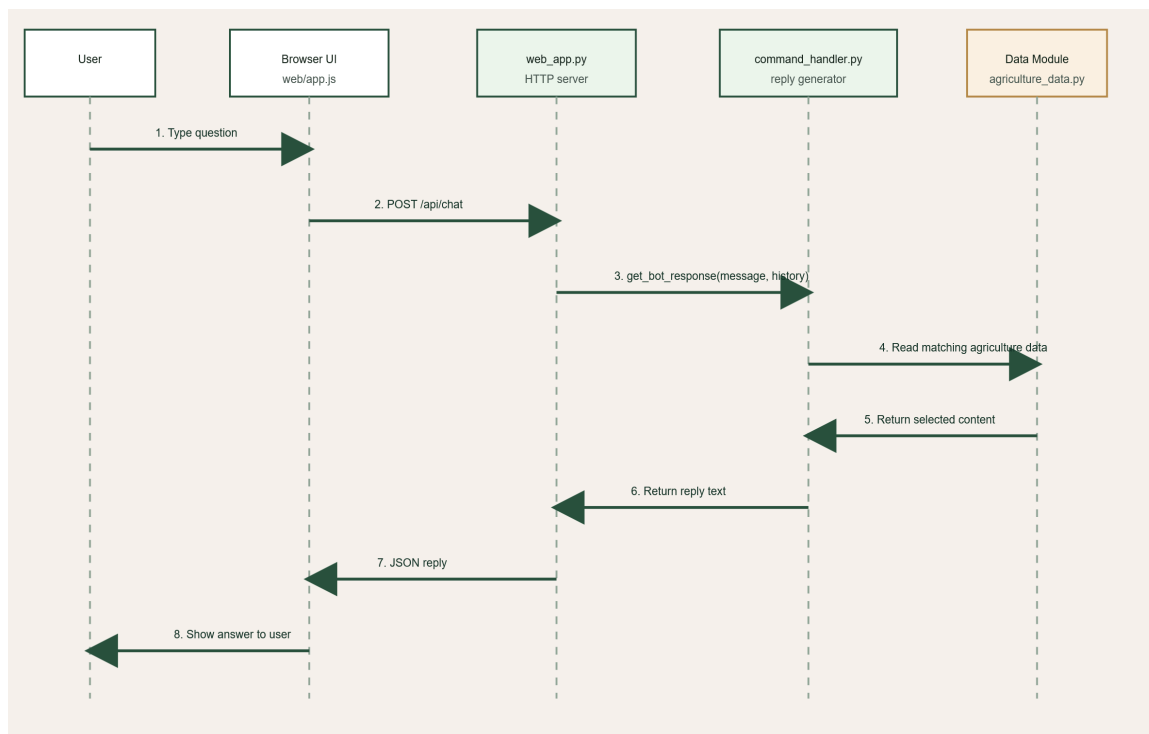
Module Diagram



2.2 Process Flow

1. User sends message via UI
2. Request sent to /api/chat
3. Server reads JSON
4. Message processed
5. Response generated
6. JSON returned
7. UI displays reply

Flow Diagram



2.3 Information Flow

Request

```
{  
  "message": "weather",  
  "history": ["crop status"]  
}
```

Response

```
{  
  "reply": "Sunny weather expected today."  
}
```

2.4 Component Design

web_app.py

- Handles routes and API
- GET: serves UI + welcome data
- POST: processes chat
- Returns 404 (invalid route), 400 (bad JSON)

command_handler.py

- Detects topic using keywords
- Generates responses

- Supports:
 - crop, soil, fertilizer (dictionary)
 - weather, tips, irrigation (random)
 - time (system time)
 - history (user inputs)

agriculture_data.py

Stores:

- crop data
- soil info
- fertilizer data
- weather updates
- tips

utils.py

- Random response
- Time function
- Console helpers

chatbot.py

- CLI loop
- Stores history
- Stops on exit command

web/app.js

Manages:

- Chat history
 - Quick actions
 - UI state
 - API calls
-

2.5 Key Design Points

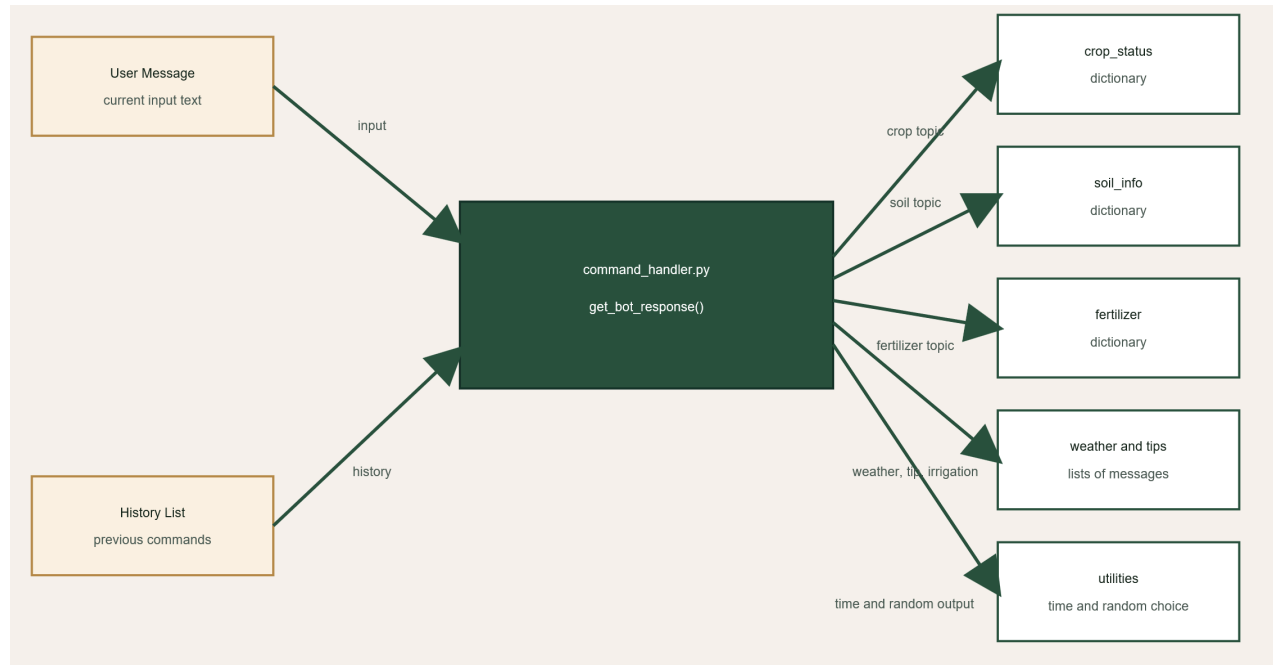
- Shared logic (web + CLI)
 - Modular structure
 - Keyword-based chatbot
 - In-memory data
 - Simple frontend state
-

2.6 API Catalogue

Endpoint	Method	Purpose
/api/welcome	GET	Welcome + quick actions
/api/chat	POST	Chat response
/	GET	Main page
/styles.css	GET	Styles
/app.js	GET	Script

3. Data Design

Data Diagram



3.1 Data Model

Data	Type	Use
crop_status	dict	crop info
soil_info	dict	soil info
fertilizer	dict	fertilizer
weather_updates	list	weather
tips	list	tips
IRRIGATION_UPDATES	list	irrigation
history	list	chat history

3.2 Data Access

- Backend calls response functions
 - Static data imported
 - No direct frontend data access
 - No database used
-

3.3 Data Retention

- No permanent storage
 - Web history resets on refresh
 - CLI history clears on exit
-

3.4 Data Migration

Currently not required. Future options:

- JSON
 - Database
 - External APIs
-

4. Interfaces

- Web (browser)
- CLI

- Backend API (HTTP + JSON)
-

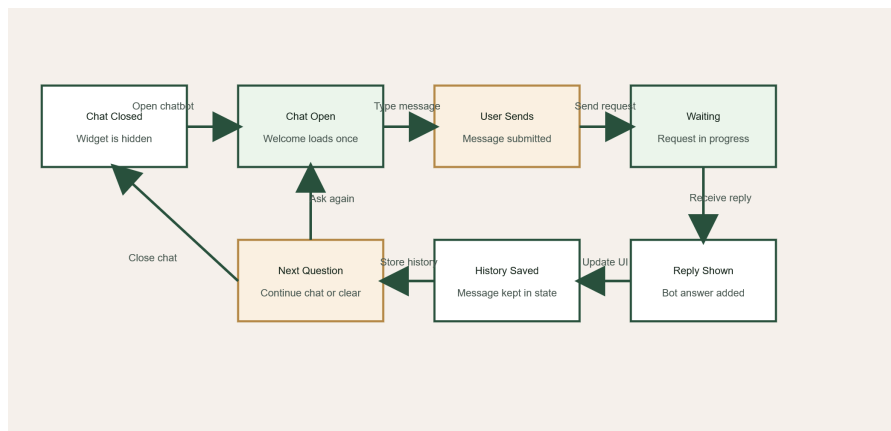
5. State Management

- Stored mainly in frontend
- Backend is stateless

State includes:

- Chat history
- Quick actions

Session Flow Diagram



6. Caching

- No dedicated caching
 - Data in memory
 - Browser may cache static files
-

7. Non-Functional Requirements

7.1 Security

- Input/output as plain text
- Invalid routes → 404
- Invalid JSON → 400

7.2 Performance

- Small data size
 - Fast keyword matching
 - Lightweight system
-

8. References

- Backend files (web_app.py, command_handler.py, etc.)
- Frontend files (index.html, app.js, styles.css)
- README.md